

COMPX341 - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

Code Quality

COMPX₃₄₁

Before we start

- Coursework 2 (preliminary) feedback
- Quotation marks in LaTeX
- Coursework 3
 - Let's take a look ...

Code Quality

COMPX₃₄₁

Overview

- Code quality – why does it matter?
- Formatting
- Naming conventions
- Comments
- Refactoring

Code quality – why does it matter?

Code quality isn't about bugs, it's about writing *good* code. Badly written code may slow down development or increase the risk of introducing bugs.

Good code should be tidy, include comments, and use universal conventions that make it easier to read and understand.

Code quality

There's a lot! But we will look at the following:

- Formatting
 - Indentation
 - Whitespace
 - Brackets
- Naming conventions
 - Class names
 - Method names
 - Variable names
- Commenting
 - Class level
 - Method level
 - Above line and inline
 - Java docs
- Refactoring

Formatting

Formatting: Indentation

In Java, indentation doesn't have to be correct for your code to run, but it does make it a lot easier to read! ...

```
boolean dictinoaryHasWord = false;
for(int i = 0; i < dictionary.length; i++){
    if(dictionary[i].equals(word)){
        dictinoaryHasWord = true;
    }
}

if(dictinoaryHasWord){
    System.out.println(testWord + " is a real word!");
} else {
    System.out.println(testWord + " is not a real word");
}
```

```
boolean dictinoaryHasWord = false;
for(int i = 0; i < dictionary.length; i++){
    if(dictionary[i].equals(word)){
        dictinoaryHasWord = true;
    }
}

if(dictinoaryHasWord){
    System.out.println(testWord + " is a real word!");
} else {
    System.out.println(testWord + " is not a real word");
}
```

Formatting: Whitespace

In Java, whitespace doesn't have to be correct for your code to run, but it does make it a lot easier to read! ...

```
String testWord = "time";
String word = testWord;
boolean includesNumerical = false;
for (int i = 0; i < word.length(); i++){
    if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){
        includesNumerical = true;
    }
}
if(includesNumerical){

    System.out.println("Sorry, there are numbers");
}
```

```
String testWord = "time";
String word = testWord;

boolean includesNumerical = false;

for (int i = 0; i < word.length(); i++){
    if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){
        includesNumerical = true;
    }
}

if(includesNumerical){
    System.out.println("Sorry, there are numbers");
}
```

Formatting: Whitespace

Whitespace is not just about newlines, but also about spaces within a line of code

```
String testWord="time";
String word=testWord;
boolean includesNumerical=false;
for (int i=0;i<word.length();i++){
    if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){
        includesNumerical=true;
    }
}
if(includesNumerical){
    System.out.println("S
}
```

```
String testWord = "time";
String word = testWord;
boolean includesNumerical = false;
for (int i = 0; i < word.length(); i++){
    if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){
        includesNumerical = true;
    }
}
System.out.println("S
    e numbers");
```

Note, spaces CAN actually introduce bugs in your code!

l.e. `current != null`
versus `current! =null`

Formatting: Brackets

In Java, the opening bracket should be on the same line as the statement. The closing bracket should be on a new line.

```
else {  
    word = word.toLowerCase();  
    word = word.replaceAll("[.!?\\"-]", "");  
  
    boolean dictinoaryHasWord = false;  
    for(int i = 0; i < dictionary.length; i++){  
        if(dictionary[i].equals(word)){  
            dictinoaryHasWord = true; } }  
  
    if(dictinoaryHasWord){  
        System.out.println(testWord + " is a real word!"); } else {  
        System.out.println(testWord + " is not a real word"); } }
```

```
else {  
    word = word.toLowerCase();  
    word = word.replaceAll("[.!?\\"-]", "");  
  
    boolean dictinoaryHasWord = false;  
    for(int i = 0; i < dictionary.length; i++){  
        if(dictionary[i].equals(word)){  
            dictinoaryHasWord = true;  
        }  
    }  
  
    if(dictinoaryHasWord){  
        System.out.println(testWord + " is a real word!");  
    } else {  
        System.out.println(testWord + " is not a real word");  
    }  
}
```

Naming conventions

Naming conventions (for Java)

- In general names should be:

- Class names should be TitleCase.
- Method names should be camelCase.
- Variable names should be camelCase.

i.e. `public class SpellChecker`

i.e. `public String convertToLower()`

i.e. `String testWord = "time";`

- Note, this applies to both single and compound-word names:

- Class names should be TitleCase.
- Method names should be camelCase.
- Variable names should be camelCase.

i.e. `public class Main`

i.e. `public void main()`

i.e. `String word = testWord;`

Naming conventions (for Java)

- Note, these naming conventions are for Java. Other languages may use different naming conventions.
- I.e. `python` often uses `under_scores` rather than `camelCase`

Naming conventions (for Java)

- Also, in general:

- Class names should be nouns. i.e. `public class Dictionary`
- Method names should be verbs (actions). i.e. `public void convertToLower()`
- Variable names should be nouns. i.e. `String word = "cat";`
- Except Booleans which should be questions. i.e. `boolean includesNumerals = false;`

Naming conventions (for Java)

- Names should be descriptive

i.e. `int s = 0;`

versus `int smallest = 0;`

- But not overly long or complex

i.e. `int theSmallestNumber = 0;`

Note, this applies to classes, methods, and variables

Commenting

Commenting

- You should comment your code at:
 - Class level
 - Method level
 - Above line – for single lines or blocks of code
 - Inline ONLY if that line is complex

Commenting

```
/**
 * This class contains all of the logic
 * to check if a word is spelled correctly
 * (i.e. exists in the dictionary provided)
 *
 * @author Jemma König
 * @version 0.0.1
 * @since 2023-05-08
 */
public class Main {

    // The dictionary of all English words
    private static String[] dictionary = {"time", "year", "people"}

    /**
     * The main entry point for this class
     * @param args. A string array of arguments (not used)
     */
    public static void main(String[] args){

        String testWord = "Time";
        String word = testWord;

        // Check whether the word contains numbers
        boolean includesNumerical = false;
        for (int i = 0; i < word.length(); i++){
            if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){
                includesNumerical = true;
            }
        }

        // If the word contains numbers, end the program here
    }
}
```

Commenting

Class level

Above line
(not inline)

Method level

```
/**
 * This class contains all of the logic
 * to check if a word is spelled correctly
 * (i.e. exists in the dictionary provided)
 *
 * @author Jemma König
 * @version 0.0.1
 * @since 2023-05-08
 */
public class Main {

    // The dictionary of all English words
    private static String[] dictionary = {"time", "year", "people"}

    /**
     * The main entry point for this class
     * @param args. A string array of arguments (not used)
     */
    public static void main(String[] args){

        String testWord = "Time";
        String word = testWord;

        // Check whether the word contains numbers
        boolean includesNumerical = false;
        for (int i = 0; i < word.length(); i++){
            if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){
                includesNumerical = true;
            }
        }

        // If the word contains numbers, end the program here
    }
}
```

Commenting

```
/**
 * The main entry point for this class
 * @param args. A string array of arguments (not used)
 */
public static void main(String[] args){

    String testWord = "Time";
    String word = testWord;

    // Check whether the word contains numbers
    boolean includesNumerical = false;
    for (int i = 0; i < word.length(); i++){
        if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){
            includesNumerical = true;
        }
    }

    // If the word contains numbers, end the program here
    // and produce an error message for the user
    if(includesNumerical){
        System.out.println("Sorry, there are numbers");
    }

    // If the word doesn't contain numbers, then continue
    else {

        // Convert to lower case and strip punctuation
        word = word.toLowerCase();
        word = word.replaceAll("[.!?\\-]", "");

        // Check if the word exists in the dictionary
        boolean dictionaryHasWord = false;
        for(int i = 0; i < dictionary.length; i++){
```

Commenting

Method level

Above line
(for blocks of
code)

Above line
(for blocks of
code)

Above line
(for blocks of
code)

Above line
(for clarification)

```
/**
 * The main entry point for this class
 * @param args. A string array of arguments (not used)
 */
public static void main(String[] args){

    String testWord = "Time";
    String word = testWord;

    // Check whether the word contains numbers
    boolean includesNumerical = false;
    for (int i = 0; i < word.length(); i++){
        if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){
            includesNumerical = true;
        }
    }

    // If the word contains numbers, end the program here
    // and produce an error message for the user
    if(includesNumerical){
        System.out.println("Sorry, there are numbers");
    }

    // If the word doesn't contain numbers, then continue
    else {

        // Convert to lower case and strip punctuation
        word = word.toLowerCase();
        word = word.replaceAll("[.!?\\-]", "");

        // Check if the word exists in the dictionary
        boolean dictionaryHasWord = false;
        for(int i = 0; i < dictionary.length; i++){
            if(word.equals(dictionary[i])){
                dictionaryHasWord = true;
            }
        }
    }
}
```

Commenting: Java Docs!

“Javadoc is a tool which comes with JDK and it is used for generating Java code documentation in HTML format from Java source code”

- To use Java Docs, you have to use comments with a predefined format
- Comments start with `/**` and end with `*/`

```
/**  
 * The main entry point for this class  
 * @param args. A string array of arguments (not used)  
 */
```

Commenting: Java Docs!

"Javadoc is a tool which comes with JDK and it is used for generating Java code documentation in HTML format from Java source code"

- There are a set of pre-defined tags that you can use
 - @param
 - @return
 - @author (for class level Java Docs)
 - @version (for class level Java Docs)
 - etc ...
- Check them out here ... https://www.tutorialspoint.com/java/java_documentation.htm

Commenting: Java Docs!

“Javadoc is a tool which comes with JDK and it is used for generating Java code documentation in HTML format from Java source code”

- To generate your Java Doc files, in the terminal, use the command:

```
javadoc ClassName.java
```

or

```
javadoc *.java
```

Commenting: Java Docs!

- Let's try it ...

Refactoring

Refactoring

- Refactoring involves restructuring your code to make it:
 - Cleaner
 - Simpler
 - More readable
 - And to align with conventions (if it doesn't already)
- You should refactor often, not at the end when everything's written!
 - I.e. each time you finish a method, refactor it!
- Warning – there is a balance. You can refactor your code so much that it becomes unreadable or unnecessarily complex.

Refactoring

```
public class Main {  
  
    private static String[] dictionary = {"time", "year", "people", "way", "day"  
  
    public static void main(String[] args){  
  
        String testWord = "time";  
        String word = testWord;  
  
        boolean includesNumerical = false;  
        for (int i = 0; i < word.length(); i++){  
            if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){  
                includesNumerical = true;  
            }  
        }  
  
        if(includesNumerical){  
            System.out.println("Sorry, there are numbers");  
        }  
  
        else {  
  
            word = word.toLowerCase();  
            word = word.replaceAll("[.!?\\"-]", "");  
  
            boolean dictionaryHasWord = false;  
            for(int i = 0; i < dictionary.length; i++){  
                if(dictionary[i].equals(word)){  
                    dictionaryHasWord = true;  
                }  
            }  
        }  
    }  
}
```

```
public class SpellChecker {  
  
    ...  
  
    public boolean checkIsWordReal(String word){  
        if(this.includesNumerical(word)){  
            return false;  
        }  
  
        word = this.convertToLower(word);  
        word = this.stripPunctuation(word);  
        return this.existsInDictionary(word);  
    }  
}
```

```
public class Main {  
  
    public static void main(String[] args){  
  
        SpellChecker spellChecker = new SpellChecker();  
        System.out.println(spellChecker.checkIsWordReal("time"));  
    }  
}
```

Finally, a word of caution

A word of caution ... this isn't everything!

- For some more, check out 'code smells'

https://en.wikipedia.org/wiki/Code_smell#:~:text=In%20computer%20programming%2C%20a%20code,%2C%20developer%2C%20and%20development%20methodology.

Also, these are not hard and fast rules!

- Different languages use different conventions
 - Different papers may have different expectations
 - In industry, different companies will use different conventions
-
- But, this is a good start!

COMPX₃₄₁ - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato